MLOps and CI/CD

With GitHub Actions

Last session

Your data is in a bucket, as Parquet, partitioned by day. An app in a container reads it.

All of it ran **by hand, on your laptop**:

- Who runs the tests before a change is merged?
- Who checks a teammate's change does not break `upload.py`?
- Who builds the image, and from which version of the code?

Today: let a machine do that, every time.

Today

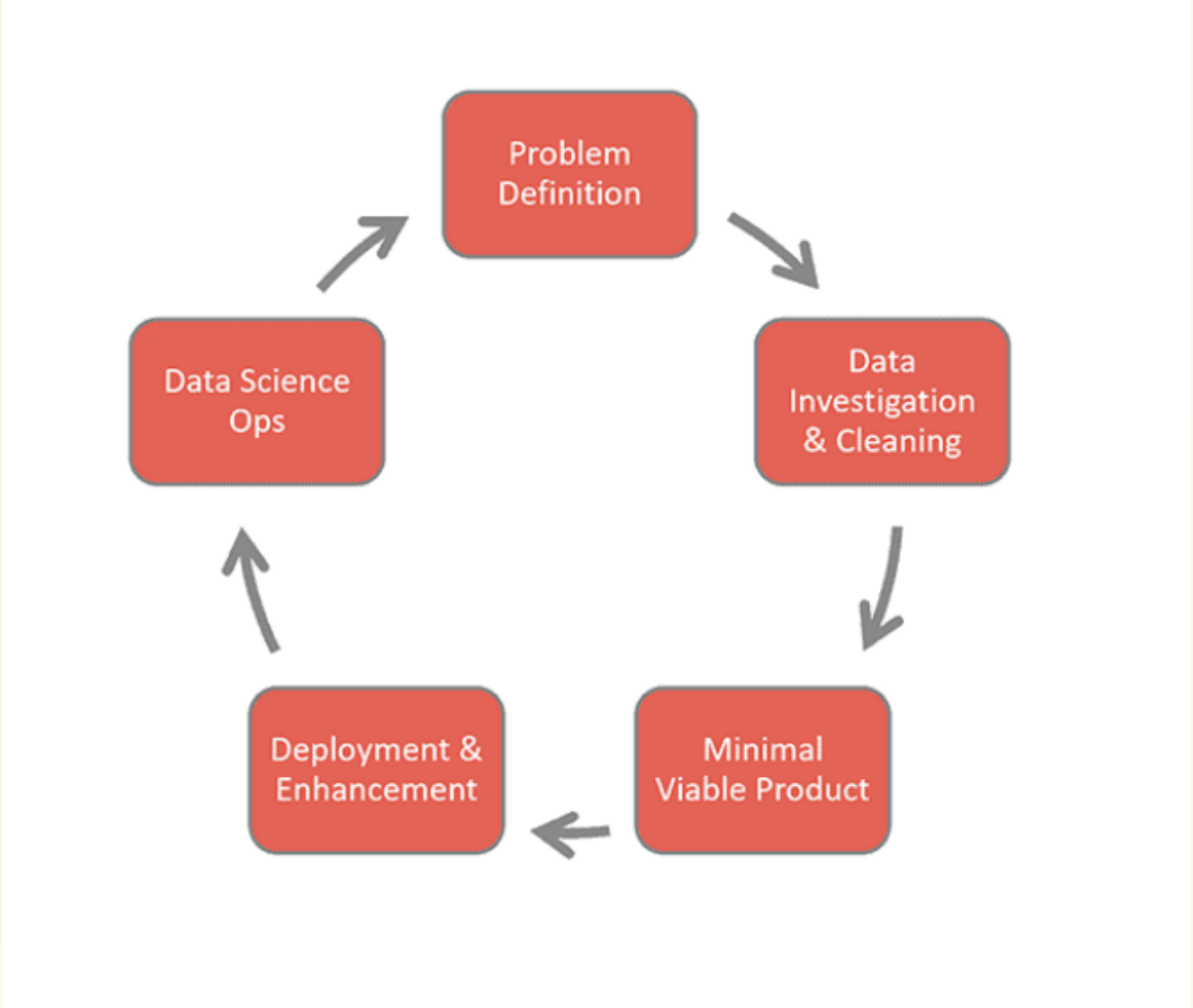
Part 1: MLOps and CI/CD. What they are and why ML makes them harder

Part 2: GitHub Actions, hands-on. Your first workflow, then tests and lint on every pull request, and a red check that blocks the merge

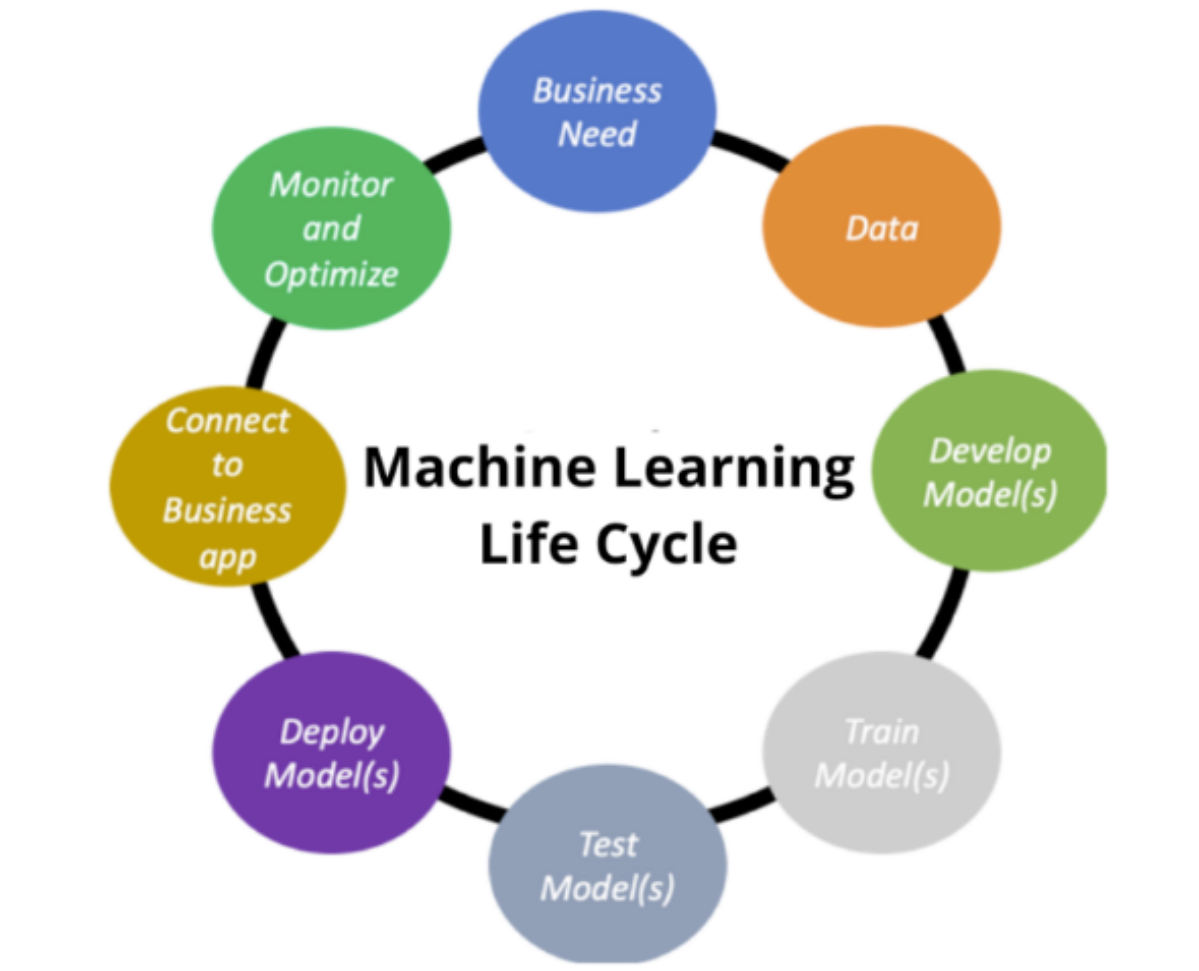
Part 1

MLOps and CI/CD

Data Science Project Lifecycle



Machine Learning Project Lifecycle

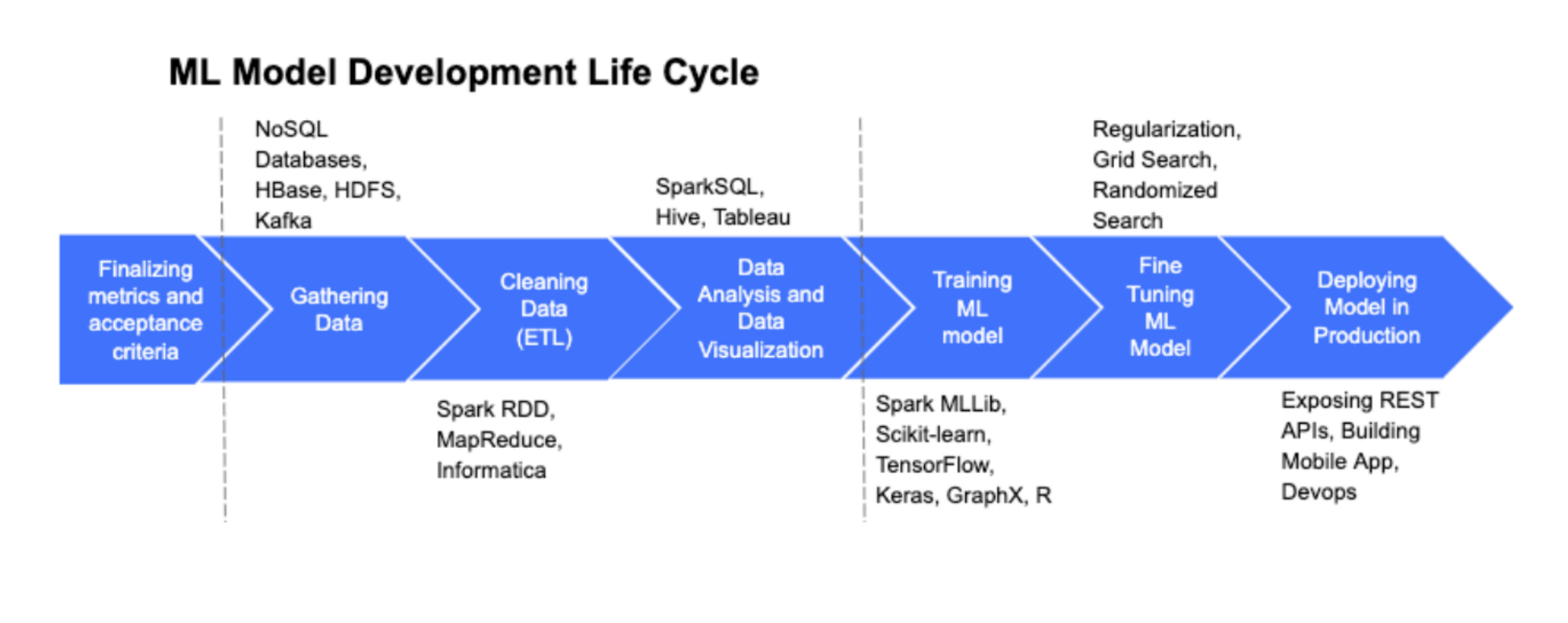


What is MLOps? ML + DevOps

Practices to get machine learning models to production and keep them working there.

- **Automation**: training, testing and deployment without manual steps
- **Versioning**: code, data and models
- **Reproducibility**: the same result on any machine
- **Monitoring**: notice when the model or the data degrade
- **Collaboration**: data scientists and engineers on the same workflow

Machine Learning Project Lifecycle



Without MLOps

- **"Works on my machine"**: the model fails somewhere else
- **Manual steps**: someone forgets one, or does them in another order
- **No tests**: a change silently breaks the data pipeline
- **No monitoring**: the model degrades and nobody notices
- **No reproducibility**: nobody can recreate last month's model

You already have some of the pieces: **Git** (session 1), **Docker** (session 2), **object storage** (session 3). Today we connect them.

What is CI/CD?

Continuous Integration (CI): every change is built and tested automatically, and merged to `main` often. Bugs show up in minutes, on the pull request, not weeks later.

Continuous Delivery/Deployment (CD): what passes CI is packaged and released automatically. No "it was built on someone's laptop".

```
Code → Build → Test → Release → Deploy → Monitor
```

CI/CD for Machine Learning

ML adds problems a normal app does not have:

- **Data**: the result depends on data, not only on code
- **Long training**: hours or days, not seconds
- **"Does it work well?"**, not only "does it run?"
- **More artifacts**: code, data, model, configuration

```
Code → Data validation → Train → Evaluate → Deploy → Monitor
```

We start with the part that is the same for any software: **test every change**.

Part 2

GitHub Actions

GitHub Actions

CI/CD built into GitHub.

- Workflows are **YAML files** in `.github/workflows/`, versioned with your code
- They run on **events**: a push, a pull request, a button, a schedule
- They run on **GitHub's machines** (runners)
- **Free** and unlimited for public repositories

Other services do the same: GitLab CI, Jenkins, CircleCI... The ideas carry over.

The vocabulary

Term	What it is
Workflow	A YAML file in `.github/workflows/`
Event	What starts it: `push`, `pull_request`…
Job	A set of steps that run on the same machine
Step	A shell command (`run`) or an action (`uses`)
Action	A reusable step someone published: `actions/checkout`
Runner	The machine that runs a job

Workflow anatomy (1) - When

```yaml
name: Hello

on:
  push:
  workflow_dispatch:
```

- `name`: what you see in the Actions tab
- `on`: **when**. On every push, and with a button (`workflow_dispatch`)

Workflow anatomy (2) - What

```yaml
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: python --version
```

- `jobs.hello`: one job, named `hello`
- `runs-on: ubuntu-latest`: a **fresh** Ubuntu virtual machine
- `uses: actions/checkout@v7`: machine starts empty. Clone your repo
- `run`: any shell command

Workflow anatomy (3) - What

```
    steps:
      - run: echo "Hello ${{ github.actor }}, this is ${{ github.ref_name }}"
```

- `${{ github.actor }}`: an **expression**. GitHub fills it in: who pushed, which branch...

The runner

A new virtual machine for every job, deleted when it finishes.

- Nothing survives between runs: install what you need **every time**
- Comes with Git, Python, Docker and many more tools preinstalled
- `ubuntu-latest`, `windows-latest`, `macos-latest`
- Public repositories: 4 CPUs, 16 GB of RAM

Same idea as a container: start from a known state, every time.

Triggers

Event	When	Typical use
`push`	Commits pushed to a branch	Tests, build on `main`
`pull_request`	A PR is opened or updated	Tests before merging
`workflow_dispatch`	Someone clicks "Run workflow"	Manual jobs, retraining
`schedule`	A cron expression	Nightly data checks

```yaml
on:
  push:
    branches: [main]
  pull_request:
```

Your first full workflow

```yaml
name: Hello

on:
  push:
  workflow_dispatch:

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: echo "Hello ${{ github.actor }}, this is ${{ github.ref_name }}"
      - run: ls -la
      - run: python --version
```

Exercise 1 - README - hello.yml

In your repository from the storage session, or a fresh copy of `ub-agile-data-science-ci` (see its README):

1. Create `.github/workflows/hello.yml`
2. Commit and push to `main`
3. Open the **Actions** tab on GitHub. Find your run and read the logs of each step
4. Click **Run workflow** to run it again by hand
5. Add a step that prints the CPUs of the machine: `nproc`

Question: `ls -la` shows your files but not `data/`. Why?

Why test data code?

Data code fails **silently**: no exception, just wrong rows or wrong numbers.

A **test** runs your code on a tiny dataset where you **know** the answer, and checks it.

pytest runs every function called `test_*`
in every file called `test_*.py`.

Some code to test

Create a new file `trips.py`:

```
import pandas as pd

# add_pickup_day: the day of each trip, as `2024-01-15`
def add_pickup_day(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["pickup_day"] = df["tpep_pickup_datetime"].dt.strftime("%Y-%m-%d")
    return df

# tip_percentage: how much people tip, as a percentage of the fare
def tip_percentage(df: pd.DataFrame) -> pd.Series:
    return df["tip_amount"] * 100 / df["fare_amount"]
```

The tests

Create `test_trips.py`: tiny DataFrames, no S3, no 3 million rows

```
import pandas as pd

from trips import add_pickup_day, tip_percentage


def test_add_pickup_day():
    trips = pd.DataFrame({"tpep_pickup_datetime": pd.to_datetime(["2024-01-15 08:30:00"])})
    assert add_pickup_day(trips)["pickup_day"].tolist() == ["2024-01-15"]
```

Continues in the next slide!!!!

The tests

Append to `test_trips.py`

```
def test_tip_percentage():
    trips = pd.DataFrame({"fare_amount": [10.0, 20.0], "tip_amount": [2.0, 3.0]})
    assert tip_percentage(trips).tolist() == [20.0, 15.0]


def test_tip_percentage_ignores_trips_without_fare():
    trips = pd.DataFrame({"fare_amount": [10.0, 0.0, -5.0], "tip_amount": [2.0, 1.0, 0.0]})
    assert tip_percentage(trips).mean() == 20.0
```

Development requirements

The app does not need pytest. Keep test tools apart:

`requirements-dev.txt`

```
-r requirements.txt
pytest==9.1.1
```

```
pip install -r requirements-dev.txt
```

Pinned, like everything else: same versions on your laptop and on CI.

The CI workflow

Create a file: `.github/workflows/ci.yml`

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
```

The CI workflow

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.14"
      - run: pip install -r requirements-dev.txt
      - run: pytest
```

`actions/setup-python`: the Python version **you** choose, not the one the runner happens to have.

Run them

```
pytest
```

```
test_trips.py ..F                                                    [100%]

_____________ test_tip_percentage_ignores_trips_without_fare _____________

    def test_tip_percentage_ignores_trips_without_fare():
        trips = pd.DataFrame({"fare_amount": [10.0, 0.0, -5.0], "tip_amount": [2.0, 1.0, 0.0]})
>       assert tip_percentage(trips).mean() == 20.0
E       assert np.float64(inf) == 20.0

FAILED test_trips.py::test_tip_percentage_ignores_trips_without_fare - assert...
========================= 1 failed, 2 passed in 0.18s =========================
```

Let CI find the bug

Do not fix it yet. Push the failing test and open a pull request:

```
git checkout -b add-tests
git add trips.py test_trips.py requirements-dev.txt .github/workflows/ci.yml
git commit -m "Add tests and CI"
git push -u origin add-tests
```

- The `test` check on the PR goes ❌
- Click **Details**: the same pytest output as on your laptop
- A step that exits with an error fails the job. The job is a **check** on the pull request

The test found a bug

Divide by a fare of 0 and pandas gives `inf`, no error.

It is not a made-up case. In January 2024: **893** trips with a fare of 0, **37,448** with a negative fare (refunds). On real data: **23.1 %** on average.

```
def tip_percentage(df: pd.DataFrame) -> pd.Series:
    paid = df[df["fare_amount"] > 0]
    return paid["tip_amount"] * 100 / paid["fare_amount"]
```

Commit, push to the **same branch**: the PR runs again ✅

Lint

A **linter** reads your code without running it: unused imports, undefined names, style. Ruff is fast.

Add `ruff==0.17.1` to `requirements-dev.txt`, then:

```
pip install -r requirements-dev.txt
ruff check .
```

```
app/app.py:4:8: PLR0402 [*] Use `from pyarrow import fs` in lieu of alias
read_remote.py:5:8: PLR0402 [*] Use `from pyarrow import fs` in lieu of alias
upload.py:4:8: PLR0402 [*] Use `from pyarrow import fs` in lieu of alias
Found 3 errors.
[*] 3 fixable with `--fix`.
```

A lint job

Add it under `jobs:` in `ci.yml`, next to `test`:

```
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.14"
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
```

Commit the fixes and the new job, push to the same branch.

Two jobs

- `lint` and `test` run **in parallel**, each on its own machine
- That is why both install the requirements
- Each job is its own check on the PR: ✅ ✅ and you can merge

Exercise 2 - README - tests and CI

1. Add `trips.py`, `test_trips.py` and `requirements-dev.txt`. Run `pytest`: red. Do not fix it yet
2. Add `ci.yml` with the `test` job
3. Push a branch `add-tests` and **open a pull request**. Watch `test` go ❌
4. Fix the bug, push again. Watch it go ✅
5. Run `ruff check .` and fix what it reports. Add the `lint` job, push
6. Both checks green: merge

Green is not enough

The checks **report**. Nothing stops you from merging a red PR.

Make them **required**: Settings → Rules → Rulesets → **New branch ruleset**

- Target: the default branch (`main`)
- **Require a pull request before merging**
- **Require status checks to pass**: add `lint` and `test`
- Enforcement status: **Active**

Now `main` only changes through a PR with green checks.

Break it on purpose

In a new branch, change the format of the day:

```
df["pickup_day"] = df["tpep_pickup_datetime"].dt.strftime("%d/%m/%Y")
```

Push it, open a PR and watch:

- `test` goes ❌. Open the log: the same pytest output as on your laptop
- The **Merge** button is blocked
- Fix it, push again to the **same branch**: the PR runs again ✅

Exercise 3 - README - required checks

1. Add the ruleset on `main` with `lint` and `test` as required checks
2. Try `git push` directly to `main`. What happens?
3. Open a PR that breaks a test. Check the merge is blocked
4. Fix it in the same PR and merge

Question: Your teammate says "it passes on my laptop". The check is red. Who is right, and how do you find out?

If you want to learn more...

Extra material

Use the functions in upload.py

`upload.py` from the storage session does the same work inline:

```
# One folder per day. A few trips have wrong dates (2002, 2009): keep January.
df = pd.read_parquet("data/trips.parquet")
jan = df[df["tpep_pickup_datetime"].between("2024-01-01", "2024-02-01")].copy()
jan["pickup_day"] = jan["tpep_pickup_datetime"].dt.strftime("%Y-%m-%d")
```

Code inside a script is hard to test. Move it into `trips.py`, test it there, and call it from the script.

keep_month

Add to `trips.py`:

```
def keep_month(df: pd.DataFrame, year: int, month: int) -> pd.DataFrame:
    pickup = df["tpep_pickup_datetime"]
    return df[(pickup.dt.year == year) & (pickup.dt.month == month)]
```

and to `test_trips.py`:

```
def test_keep_month_drops_wrong_dates():
    trips = pd.DataFrame({"tpep_pickup_datetime": pd.to_datetime(
        ["2002-12-31 23:00:00", "2024-01-15 08:30:00", "2009-01-01 00:10:00"])})
    assert len(keep_month(trips, 2024, 1)) == 1
```

upload.py, now

```
from trips import add_pickup_day, keep_month

# One folder per day. A few trips have wrong dates (2002, 2009): keep January.
df = pd.read_parquet("data/trips.parquet")
jan = add_pickup_day(keep_month(df, 2024, 1))
```

Same result: 2,964,606 trips in 31 days.

Now CI tests the code that writes your bucket. Open it as a PR.